

Unit -1: Introduction, HTML & XML

Lecture 1

1.1 Introduction and Web development strategies

Web consists of billions of clients and server connected through wires and wireless networks. The web clients make requests to web server. The web server receives the request, finds the resources and returns the response to the client. When a server answers a request, it usually sends some type of content to the client. The client uses web browser to send request to the server. The server often sends response to the browser with a set of instructions written in HTML (Hypertext Mark-up Language). All browsers know how to display HTML page to the client.

1.1.1 What is WWW?

The World Wide Web (WWW), often called the Web, is a system of interconnected WebPages and information that you can access using the Internet. It was created to help people share and find information easily, using links that connect different pages together. The Web allows us to browse websites, watch videos, shop online, and connect with others around the world through our computers and phones.

- WWW stands for World Wide Web and is commonly known as the Web.
- The WWW was started by CERN in 1989.
- WWW is defined as the collection of different websites around the world, containing different information shared via local servers(or computers).
- Web pages are linked together using hyperlinks which are HTML-formatted and, also referred to as hypertext, these are the fundamental units of the Internet and are accessed through Hypertext Transfer Protocol(HTTP).
- Such digital connections, or links, allow users to easily access desired information by connecting relevant pieces of information.
- The benefit of hypertext is it allows you to pick a word or phrase from the text and click on other sites that have more information about it.

1.1.2 History of the WWW(WEB)

The World Wide Web was created in 1989 by Tim Berners-Lee and his team at CERN in Geneva, Switzerland. They made a standard way for computers to communicate, called HyperText Transfer Protocol (HTTP). Their text-based Web browser was released in January 1992.

The Web became popular quickly with the creation of a browser called Mosaic. Developed by Marc Andreessen and his team at the University of Illinois, Mosaic was released in September 1993. It allowed users to click on links and navigate easily. In April 1994, Andreessen co-founded Netscape Communications Corporation and released Netscape Navigator in December 1994, which became the

top browser. That same year, Book Link Technologies introduced Internetworks, the first browser with tabs.

By the mid-1990s, millions of people were using the Web. Microsoft entered the scene with Internet

Explorer (IE) in 1995, based on Mosaic, and included it in Windows 95. This made IE the most popular browser, reducing competition like Netscape.

Apple launched Safari in 2003 as the default browser for Macintosh computers, and later for iPhones (2007) and iPads (2010). Safari 2.0 in 2005 introduced Private Browsing, which did not save website history, downloaded files, or personal information.

1.1.3 The Web Extends

Only a few people had access to the first Web browser, which ran on a NeXT computer. To make it more accessible, Nicola Pellow created a simpler browser called the 'line-mode' browser. This browser could run on any system.

In 1991, Tim Berners-Lee released his WWW software. It included the 'line-mode' browser, Web server software, and a library for developers. In March 1991, the software was available to CERN colleagues. By August 1991, Berners-Lee announced the WWW software on Internet newsgroups, and interest in the project spread worldwide.

1.1.3.1 Why use Web?

- When it seems that social media rules the Internet, you might ask yourself, "Do I need a website?" The answer is yes, and I'm here to tell you why.
 - In 2022, 70-80% of people were researching companies online before visiting it or making a purchase, and the same percentage of customers could be lost to small businesses without a website. And when the world suddenly required more online presence than real-life presence the following year, having your own website became even more crucial.
 - If you're not convinced yet, and you're curious about the advantages of websites over social media profiles, read on! Here are five ultimate reasons why you need a website in 2024.
1. Having a website makes you look professional and increases trust.
 2. A business website will bring you more customers and increase conversions.
 3. Creating your own website is *much* easier and less expensive than you expect.
 4. A website gives you full control over the medium.
 5. Websites are the center of all marketing efforts.

1.1.3.2 What is Web Application?

A website is a collection of static files(webpages) such as HTML pages, images, graphics etc. A Web application is a web site with dynamic functionality on the server. Google, Facebook, Twitter are examples of web applications.

1.2 Web development Strategies

- i. Responsive Web Design: Make your site look good on everything, from computers to phones.

- ii. Performance Optimization: Make your site fast so people don't get bored waiting.
- iii. User-Centered Design: Design your site for people, not robots.
- iv. Security Measures: Keep your site safe from sneaky online troublemakers.
- v. Cross-Browser Compatibility: Ensure your site works on all web browsers.
- vi. Mobile-First Development: Think about phones when building your site.
- vii. Content Management and SEO: Write good stuff and help people find it.

1.2.1 Web Technology can be Classified into the Following Sections:

- **World Wide Web (WWW):** The World Wide Web is based on several different technologies: Web browsers, Hypertext Markup Language (HTML), and Hypertext Transfer Protocol (HTTP).
- **Web Browser:** The web browser is an application software to explore www (World Wide Web). It provides an interface between the server and the client and requests to the server for web documents and services.
- **Web Server:** Web server is a program which processes the network requests of the users and serves them with files that create web pages. This exchange takes place using Hypertext Transfer Protocol (HTTP).
- **Web Pages:** A webpage is a digital document that is linked to the World Wide Web and viewable by anyone connected to the internet has a web browser.
- **Web Development:** Web development refers to the building, creating, and maintaining of websites. It includes aspects such as web design, web publishing, web programming, and database management. It is the creation of an application that works over the internet i.e. websites.

1.2.2 What is IP Addressing?

An IP address represents an Internet Protocol address. A unique address that identifies the device over the network. It is almost like a set of rules governing the structure of data sent over the Internet or through a local network. An IP address helps the Internet to distinguish between different routers, computers, and websites. It serves as a specific machine identifier in a specific network and helps to improve visual communication between source and destination.

Lecture 2

1.3 Protocols Governing Web

1.3.1 What is protocol?

A protocol is a set of rules that explain how to conduct oneself in formal situations or how to transmit data between entities. Protocols can be implemented by software or hardware, but they are not software or hardware themselves

1.3.2 Why Use Protocol?

a. Communication

Protocols allow devices to communicate with each other by establishing rules for how data is structured and exchanged. This allows devices to communicate even if they have different software, hardware, or internal processes. For example, the Internet Protocol (IP) allows computers to communicate with each other, even if they use different software and hardware.

b. Transparency

Protocols can make processes transparent, which can help to avoid bias and duplication.

c. Peer review

Protocols can allow for peer review by reviewers with expertise in the area.

d. Coding

Protocols can be used in coding to improve code. For example, statically typed languages check types at compile time, so issues with types are avoided at runtime.

Internet Protocols are of different types having different uses. These are

mentioned below:

1. TCP/IP(Transmission Control Protocol/ Internet Protocol)

These are a set of standard rules that allows different types of computers to communicate with each other. The IP protocol ensures that each computer that is connected to the Internet is having a specific serial number called the IP address. TCP specifies how data is exchanged over the internet and how it should be broken into IP packets. It also makes sure that the packets have information about the source of the message data, the destination of the message data, the sequence in which the message data should be re-assembled, and checks if the message has been sent correctly to the specific destination. The TCP is also known as a connection-oriented protocol.

For more details, please refer [TCP/IP Model](#) article.

2. SMTP(Simple Mail Transfer Protocol)

These protocols are important for sending and distributing outgoing emails. This protocol uses the header of the mail to get the email id of the receiver and enters the mail into the queue of outgoing mail. And as soon as it delivers the mail to the receiving email id, it removes the email from the outgoing list. The message or the electronic mail may consider the text, video, image, etc. It helps in setting up some communication server rules.

3. PPP(Point-to-Point Protocol)

It is a communication protocol that is used to create a direct connection between two communicating devices. This protocol defines the rules using which two devices will authenticate with each other and exchange information with each other.

For example, A user connects his PC to the server of an Internet Service Provider and also uses PPP. Similarly, for connecting two routers for direct communication it uses PPP.

4. FTP (File Transfer Protocol)

This protocol is used for transferring files from one system to the other. This works on a client-server model. When a machine requests for file transfer from another machine, the FTO sets up a connection between the two and authenticates each other using their ID and Password. And, the desired file transfer takes place between the machines.

5. SFTP(Secure File Transfer Protocol)

SFTP which is also known as SSH FTP refers to File Transfer Protocol (FTP) over Secure Shell (SSH) as it encrypts both commands and data while in transmission. SFTP acts as an extension to SSH and encrypts files and data then sends them over a secure shell data stream. This protocol is used to remotely connect to other systems while executing commands from the command line.

6. HTTP (HyperText Transfer Protocol)

This protocol is used to transfer hypertexts over the internet and it is defined by the www(world wide web) for information transfer. This protocol defines how the information needs to be formatted and transmitted. And, it also defines the various actions the web browsers should take in response to the calls made to access a particular web page. Whenever a user opens their web browser, the user will indirectly use HTTP as this is the protocol that is being used to share text, images, and other multimedia files on the World Wide Web.

Note: Hypertext refers to the special format of the text that can contain

links to other texts. **7. HTTPS (HyperText Transfer Protocol Secure)**

HTTPS is an extension of the Hypertext Transfer Protocol (HTTP). It is used for secure communication over a computer network with the SSL/TLS protocol for encryption and authentication. So, generally, a website has an HTTP protocol but if the website is such that it receives some sensitive information such as credit card details, debit card details, OTP, etc then it requires an SSL certificate installed to make the website more secure. So, before entering any sensitive information on a website, we should check if the link is HTTPS or not. If it is not HTTPS then it may not be secure enough to enter sensitive information.

8. TELNET (Terminal Network)

TELNET is a standard TCP/IP protocol used for virtual terminal service given by

ISO. This enables one local machine to connect with another. The computer which is being connected is called a remote computer and which is connecting is called the local computer. TELNET operation lets us display anything being performed on the remote computer in the local computer. This operates on the client/server principle. The local computer uses the telnet client program whereas the remote computer uses the telnet server program.

9. POP3 (Post Office Protocol 3)

POP3 stands for Post Office Protocol version 3. It has two Message Access Agents (MAAs) where one is client MAA (Message Access Agent) and another is server MAA (Message Access Agent) for accessing the messages from the mailbox. This protocol helps us to retrieve and manage emails from the mailbox on the receiver mail server to the receiver's computer. This is implied between the receiver and the receiver mail server. It can also be called a one-way client-server protocol. The POP3 WORKS ON THE 2 PORTS I.E. PORT 110 AND PORT 995.

1.4 What is the need of DNS?

Every host is identified by the IP address but remembering numbers is very difficult for people. Also the IP addresses are not static therefore a mapping is required to change the domain name to the IP address. So DNS is used to convert the domain name of the websites to their numerical IP address.

Lecture 3

1.5 Connecting to Internet

The internet offers a range of services to its consumers. We can upload and download the files/ data via the internet as it is a pool of knowledge. We can access or obtain information as needed. It is quite popular because of the variety of services available on the Internet. Web services have grown in popularity as a result of these offerings.

1.5.1 Introduction to Internet services and tools :

To access/exchange a large amount of data such as software, audio clips, video clips, text files, other documents, etc., we need internet services. You must use an Internet service to connect to the Internet. Data can be sent from Internet servers to your machine via Internet service. Some of the commonly used internet services are :

1. Communication Services: To exchange data/information among individuals or organizations, we need communication services. Following are some of the common communication services:

- **IRC(Internet Relay Chat):** Subscribers can communicate in real-time by connecting numerous computers in public spaces called channels.

- **VoIP:** It stands for Voice over Internet Protocol, which describes how to make and receive phone calls over the internet. A larger number of people believe VoIP is a viable alternative to traditional landlines. VoIP (Voice over Internet Protocol) is a technique that helps us make voice calls via the Internet rather than over a traditional (or analog) phone line. Some VoIP services may let you call only other VoIP users, while others may let you call anyone with a phone number, including long-distance, mobile, and local/international lines.

If you have an internet connection you can easily call anyone without using a local phone service because VoIP solutions are based on open standards, they can be used on any computer. More than just setting up calls is what VoIP service providers do. Outgoing and incoming calls are routed through existing telephone networks by them.

- **List Server (LISTSERV):** Delivers a group of email recipients' content-specific emails.
- **E-Mail:** Used to send electronic mail via the internet. It is a paperless method for sending text, images, documents, videos, etc from one person to another via the internet.
- **User Network (USENET):** It hosts newsgroups and message boards on certain topics, and it is mostly run by volunteers.
- **Telnet:** It's used to connect to a remote computer that's connected to the internet.
- **Video Conferencing:** Video conferencing systems allow two or more people who are generally in different locations to connect live and visually. Live video conferencing services are necessary for simulating face-to-face talks over the internet. The system can vary from very simple to complex, depending on the live video conferencing vendors. A live video-based conference involves two or more individuals in separate locations utilizing video-enabled devices and streaming voice, video, text, and presentations in real-time via the internet. It allows numerous people to connect and collaborate face to face over large distances. Tools available for this purpose are Zoom, FreeConference, Google Hangouts, Skype, etc.

2. Information Retrieval Services: It is the procedure for gaining access to information/data stored on the Internet. Net surfing or browsing is the process of discovering and obtaining information from the Internet. When your computer is linked to the Internet, you may begin retrieving data. To get data, we need a piece of software called a Web browser. A print or computer-based information retrieval system searches for and locates data in a file, database, or other collection of data. Some sites are:

- **www.geeksforgeeks.org:** Free tutorials, millions of articles, live, online, and classroom courses, frequent coding competitions, industry expert webinars, internships, and job possibilities are all available. A computer-based system

for searching and locating data in a file, database, or another source.

- **www.crayola.com:** It includes advice for students, parents, and educators on how to be more creative.

3. File Transfer: The exchange of data files across computer systems is referred to as file transfer. Using the network or internet connection to transfer or shift a file from one computer to another is known as file transfer. To share, transfer, or send a file or logical data item across several users and/or machines, both locally and remotely, we use file transfer. Data files include – documents, multimedia, pictures, text, and PDFs and they can be shared by uploading or downloading them. To retrieve information from the internet, there are various services available such as:

- **Gopher:** A file retrieval application based on hierarchical, distributed menus that is simple to use.
- **FTP (File Transfer Protocol):** To share, transfer, or send a file or logical data item across several users and/or machines, both locally and remotely.
- **Archie:** A file and directory information retrieval system that may be linked to FTP

4. Web services: Web services are software that uses defined messaging protocols and are made accessible for usage by a client or other web-based programs through an application service provider's web server. Web services allow information to be exchanged across web-based applications. Using Utility Computing, web services can be provided.

5. World Wide Web: The internet is a vast network of interconnected computers. Using this network, you can connect to the World Wide Web (abbreviated as 'www' or 'web') is a collection of web pages. The web browser lets you access the web via the internet.

6. Directory Services: A directory service is a set of software that keeps track of information about your company, customers, or both. Network resource names are mapped to network addresses by directory services. A directory service provides users and administrators with full transparent access to printers, servers, and other network devices. The directory services are :

- **DNS (Domain Name System):** This server provides DNS. The mappings of computer hostnames and other types of domain names to IP addresses are stored on a DNS server.
- **LDAP (Lightweight Directory Access Protocol):** It is a set of open protocols that are used for obtaining network access to stored data centrally. It is a cross-platform authentication protocol for directory services and also allows users to interact with other directory services servers.

7. Automatic Network Address Configuration: Automatic Network Addressing assigns a unique IP address to every system in a network. A DHCP Server is a network

server that is used to assign IP addresses, gateways, and other network information to client devices. It uses Dynamic Host Configuration Protocol as a common protocol to reply to broadcast inquiries from clients.

8. Network Management Services: Network management services are another essential internet service that is beneficial to network administrators. Network management services aid in the prevention, analysis, diagnosis, and resolution of connection problems. The two commands related to this are:

- **ping:** The ping command is a Command Prompt command that is used to see if a source can communicate with a specific destination & get all the possible paths between them.
- **tracert:** To find the path between two connections, use the trace route command.

9. Time Services: Using facilities included in the operating system, you may set your computer clock via the Internet. Some services are :

- **Network Time Protocol (NTP):** It is a widely used internet time service that allows you to accurately synchronize and adjust your computer clock.
- **The Simple Network Time Protocol (SNTP):** It is a time-keeping protocol that is used to synchronize network hardware. When a full implementation of NTP is not required, then this simplified form of NTP is typically utilized.

10. Usenet: The 'User's Network' is also known as Usenet. It is a network of online discussion groups. It's one of the first networks where users may upload files to news servers and others can view them.

11. News Group: It is a lively Online Discussion Forum that is easily accessible via Usenet. Each newsgroup contains conversations on a certain topic, as indicated by the newsgroup name. Users can use newsreader software to browse and follow the newsgroup as well as comment on the posts. A newsgroup is a debate about a certain topic made up of notes posted to a central Internet site and distributed over Usenet, a global network of news discussion groups. It uses Network News Transfer Protocol (NNTP).

12. E-commerce: Electronic commerce, also known as e-commerce or e-Commerce, is a business concept that allows businesses and individuals to buy and sell goods through the internet. Example: Amazon, Flipkart, etc. websites/apps

.

Lecture 4

1.6 Client Server Computing

In client server computing, the clients requests a resource and the server provides that resource. A server may serve multiple clients at the same time while a client is in contact with only one server. Both the client and server usually communicate via a computer network but sometimes they may reside in the same system.

An illustration of the client server system is given as follows –

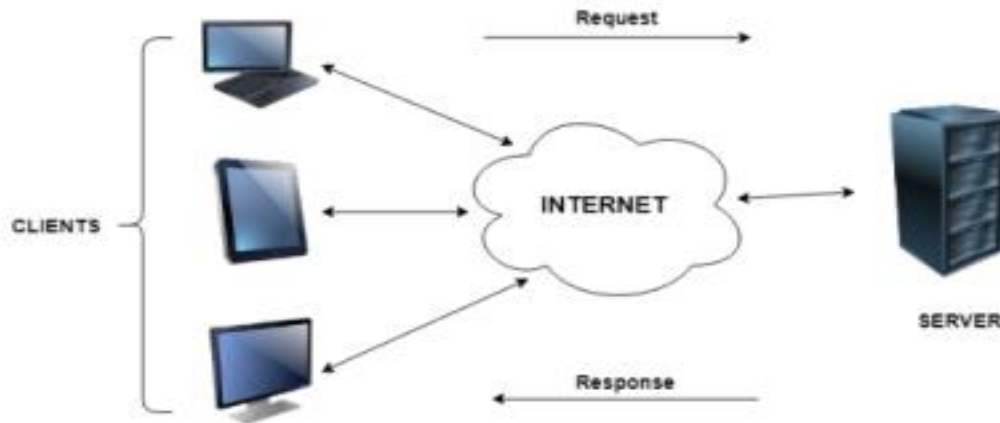


Figure 1.1 client server computing

1.6.1 Characteristics of Client Server Computing

- The client server computing works with a system of request and response. The client sends a request to the server and the server responds with the desired information.
- The client and server should follow a common communication protocol so they can easily interact with each other. All the communication protocols are available at the application layer.
- A server can only accommodate a limited number of client requests at a time. So it uses a system based on priority to respond to the requests.
- Denial of Service attacks hinder a server's ability to respond to authentic client requests by inundating it with false requests.

- An example of a client server computing system is a web server. It returns the web pages to the clients that requested them.

1.6.2 Difference between Client Server Computing and Peer to Peer Computing .

The major differences between client server computing and peer to peer computing are as follows:

In client server computing, a server is a central node that services many client nodes. On the other hand, in a peer to peer system, the nodes collectively use their resources and communicate with each other.

In client server computing the server is the one that communicates with the other nodes. In peer to peer to computing, all the nodes are equal and share data with each other directly. Client Server computing is believed to be a subcategory of the peer to peer computing.

1.6.3 Advantages of Client Server Computing

The different advantages of client server computing are –

- All the required data is concentrated in a single place i.e. the server. So it is easy to protect the data and provide authorisation and authentication.
- The server need not be located physically close to the clients. Yet the data can be accessed efficiently.
- It is easy to replace, upgrade or relocate the nodes in the client server model because all the nodes are independent and request data only from the server.
- All the nodes i.e clients and server may not be build on similar platforms yet they can easily facilitate the transfer of data.

1.6.4 Disadvantages of Client Server Computing

The different disadvantages of client server computing are –

- If all the clients simultaneously request data from the server, it may get overloaded. This may lead to congestion in the network.
- If the server fails for any reason, then none of the requests of the clients can be fulfilled. This leads of failure of the client server

network.

- The cost of setting and maintaining a client server model are quite high.

1.7 What is HTTP?

The Hypertext Transfer Protocol (HTTP) is designed to enable communications between clients and servers.

HTTP works as a request-response protocol between a client and server.

Example: A client (browser) sends an HTTP request to the server; then the server returns a response to the client. The response contains status information about the request and may also contain the requested content.

1.7.1 GET and POST

The following table compares the two HTTP methods: GET and POST.

	GET	POST
BACK button/Reload	Harmless	Data will be re-submitted (the browser should alert the user that the data are about to be re-submitted)
Bookmarked	Can be bookmarked	Cannot be bookmarked
Cached	Can be cached	Not cached
Encoding type	application/x-www-form-urlencoded	application/x-www-form-urlencoded or multipart/form-data. Use multipart encoding for binary data
History	Parameters remain in browser history	Parameters are not saved in browser history
Restrictions on data length	Yes, when sending data, the GET method adds the data to the URL; and the length of a URL is limited (maximum URL length is 2048 characters)	No restrictions
Restrictions on data type	Only ASCII characters allowed	No restrictions. Binary data is also allowed

Security	GET is less secure compared to POST because data sent is part of the URL	POST is a little safer than GET because the parameters are not stored in browser history or in web server logs
	Never use GET when sending passwords or other sensitive information!	
Visibility	Data is visible to everyone in the URL	Data is not displayed in the URL

Lecture-5

1.8. What Is HTML?

HTML stands for Hypertext Mark-up Language, and it's a code that's used to structure and define the content of a web page:

History and Evolution

Here you will see the evolution of HTML over the past couple of decades. The major upgrade was done in HTML5 in 2012.

Year Progress

- 1991 Tim Berners-Lee created HyperText Markup Language but it was not officially released.
- 1993 Tim Berners-Lee created the first version of HTML that was published and available to the public.
- 1995 HTML 2.0 was released with a few additional features along with the existing features.
- 1997 There was an attempt to extend HTML with HTML 3.0, but it was replaced by the more practical HTML 3.2.
- 1998 The W3C (World Wide Web Consortium) decided to shift focus to an XML-based HTML equivalent called XHTML.

- 1999 HTML 4.01, which became an official standard in December 1999, was the most widely used version in the early 2000s.
- 2000 XHTML 1.0, completed in 2000, was a combination of HTML4 in XML.
- 2003 The introduction of XForms reignited interest in evolving HTML itself rather than replacing it with new technologies. This new theory recognized that XML was better suited for new technologies like RSS and Atom, while HTML remained the cornerstone of the web.
- 2004 A W3C workshop took place to explore reopening HTML's evolution. Mozilla and Opera jointly presented the principles that later influenced HTML5.
- 2006 The W3C expressed interest in HTML5 development and formed a working group to collaborate with the WHATWG. The W3C aimed to publish a "finished" HTML5 version, whereas the WHATWG focused on a Living Standard, continuously evolving HTML.
- 2012 HTML5 can be seen as an extended version of HTML 4.01, which was officially published in 2012.

Evolution of HTML Features: From HTML 1.2 to HTML 5

With the introduction of new versions of HTML, support for additional features was added, and the user experience was enhanced. The following table shows the features introduced in later versions of HTML:

Type of Content	HTML 1.2	HTML 4.01	HTML 5	Description
Image	Yes	Yes	Yes	The img tag allows to add images to HTML document
Paragraph	Yes	Yes	Yes	Paragraph element in HTML is used to represent a paragraph of text on a webpage.
Heading	Yes	Yes	Yes	Heading are used in HTML to define variable length headings. (h1 to h6)
Address	Yes	Yes	Yes	Address element in HTML is used to contain contact information of user.
Anchor	Yes	Yes	Yes	Anchor tag is used to define hyperlink in webpage.
List	Yes	Yes	Yes	List is used in HTML to display list of related items.

<u>Table</u>	No	Yes	Yes	Table is used to organize data into rows and columns
<u>Style</u>	No	Yes	Yes	Style is used to add CSS styling to webpage
<u>Script</u>	No	Yes	Yes	Script is used to add JavaScript to HTML.
<u>Audio</u>	No	No	Yes	Enables introduction of audio to webpage
<u>Video</u>	No	No	Yes	Enables introduction of video to webpage.
<u>Canvas</u>	No	No	Yes	Enables introduction of graphics elements to webpage.

What is an html File?

HTML is a format that tells a computer how to display a web page. The documents themselves are plain text files with special "tags" or codes that a web browser uses to interpret and display information on your computer screen.

An HTML file is a text file containing small mark-up tags

The markup tags tell the Web browser how to display the page. An HTML file must have an .htm or .html file extension.

What are HTML tags?

- HTML tags are used to mark-up HTML elements
- HTML tags are surrounded by the two characters < and >
- The surrounding characters are called angle brackets
- HTML tags normally come in pairs like and
- The first tag in a pair is the start tag, the second tag is the end tag • the text between the start and end tags is the element content
- HTML tags are not case sensitive, means the same as

1.8.1 Logical vs. Physical Tags

In HTML there are both logical tags and physical tags. Logical tags are designed to describe (to the browser) the enclosed text's meaning. An example of a logical tag is the tag. By placing text in between these tags you are telling the browser that the text has some greater importance. By default all browsers make the text appear bold when in between the and tags.

Physical tags on the other hand provide specific instructions on how to display the text they enclose. Examples of physical tags include:

- `<b>`: Makes the text bold.
- `<big>`: Makes the text usually one size bigger than what is around it.
- `<i>`:

Makes text italic

Physical tags were invented to add style to HTML pages because style sheets were not around, though the original intention of HTML was to not have physical tags. Rather than use physical tags to style your HTML pages, you should use style sheets.

1.8.2 HTML Elements

Remember the HTML example from the previous page:

```
<html>
<head>
<title>My First Webpage</title>
</head>
<body>
This is my first homepage. <b>This text is bold</b>
</body>
</html>
```

This is an HTML element:

```
<b>This text is bold</b>
```

The HTML element begins with a start tag: `<b>`

The content of the HTML element

is: This text is bold

The HTML element ends with an end tag: `</b>`

The purpose of the `<b>` tag is to define an HTML element that should be displayed

as bold. This is also an HTML element:

```
<body>
```

This is my first homepage. `<b>This text is bold</b>`

```
</body>
```

This HTML element starts with the start tag `<body>`, and ends with the end tag `</body>`. The purpose of the `<body>` tag is to define the HTML element that contains the body of the HTML document.

1.8.3 Nested Tags

You may have noticed in the example above, the <body>tag also contains other tags, like the tag. When you enclose an element in with multiple tags, the last tag opened should be the first tag closed. For example:

```
<p><b><em>This is NOT the proper way to close nested tags.</p></em></b>
```

```
<p><b><em>This is the proper way to close nested tags.</em></b></p>
```

Note: It doesn't matter which tag is first, but they must be closed in the proper order.

1.8.4 Why Use Lowercase Tags?

You may notice we've used lowercase tags even though I said that HTML tags are not case sensitive.

 means the same as . The World Wide Web Consortium (W3C), the group responsible for developing web standards, recommends lowercase tags in their HTML 4 recommendation, and XHTML (the next generation HTML) requires lowercase tags.

1.8.5 Tag Attributes

Tags can have attributes. Attributes can provide additional information about the HTML elements on your page. The <tag> tells the browser to do something, while the attribute tells the browser how to do it. For instance, if we add the bgcolor attribute, we can tell the browser that the background color of your page should be blue, like this:

```
<body bgcolor="blue">.
```

This tag defines an HTML table: <table>. With an added border attribute, you can tell the browser that the table should have no borders: <table border="0">. Attributes always come in name/value pairs like this: name="value". Attributes are always added to the start tag of an HTML element and the value is surrounded by quotes.

Quote Styles, "red" or 'red'?

Attribute values should always be enclosed in quotes. Double style quotes are the most common, but single style quotes are also allowed. In some rare situations, like when the attribute value itself contains quotes, it is necessary to use single quotes:

```
name='George "machine Gun" Kelly'
```

Note: Some tags we will discuss are deprecated, meaning the World Wide Web Consortium (W3C) the governing body that sets HTML, XML, CSS, and other technical standards decided those tags and attributes are marked for deletion in future versions of HTML and XHTML. Browsers should continue to support deprecated tags and attributes, but eventually these tags are likely to become obsolete and so future support cannot be guaranteed.

1.8.6 Basic HTML Tags

The most important tags in HTML are tags that define headings, paragraphs and line breaks.

Basic HTML Tags

Tag	Description
<html>	Defines an HTML document
<body>	Defines the document's body
<h1> to <h6>	Defines header 1 to header 6
<p>	Defines a paragraph
 	Inserts a single line break
<hr>	Defines a horizontal rule
<!-->	Defines a comment

Headings

Headings are defined with the <h1>to <h6>tags. <h1>defines the largest heading while <h6>defines the smallest.

<h1>This is a heading</h1>

<h2>This is a heading</h2>

<h3>This is a heading</h3>

<h4>This is a heading</h4>

<h5>This is a heading</h5>

<h6> This is a heading</h6>

HTML automatically adds an extra blank line before and after a heading. A useful heading attribute is align.

<h5 align="left">I can align headings </h5>

<h5 align="center">This is a
centered heading </h5>

<h5 align="right">This is a heading aligned to the right </h5>

Paragraphs

Paragraphs are defined with the <p> tag. Think of a paragraph as a block of text. You can use the align attribute with a paragraph tag as well.

```
<p align="left">This is a paragraph</p>
```

```
<p align="center">this is another  
paragraph</p>
```

Important: You must indicate paragraphs with <p> elements. A browser ignores any indentations or blank lines in the source text. Without <p> elements, the document becomes one large paragraph. HTML automatically adds an extra blank line before and after a paragraph.

Line Breaks

The
tag is used when you want to start a new line, but don't want to start a new paragraph. The
tag forces a line break wherever you place it. It is similar to single spacing in a document.

Horizontal Rule

The <hr>element is used for horizontal rules that act as dividers between sections, like this:
The horizontal rule does not have a closing tag. It takes attributes such as align and width.

1.8.7 Comments in HTML

The comment tag is used to insert a comment in the HTML source code. A comment can be placed anywhere in the document and the browser will ignore everything inside the brackets. You can use comments to write notes to yourself, or write a helpful message to someone looking at your sourcecode.

This Code	Would Display
<pre><p> This html comment would <!-- This is a comment --> be displayed like this.</p></pre>	This HTML comment would be displayed like this.

Notice you don't see the text between the tags <!-- and -->. If you look at the source code, you would see the comment. To view the source code for this page, in your browser window, select **View** and then select **Source**.

Note: You need an exclamation point after the opening bracket <!-- but not before the closing bracket -->.

1.8.8 HTML Lists

HTML provides a simple way to show unordered lists (bullet lists) or ordered lists (numbered lists).

Unordered Lists

An unordered list is a list of items marked with bullets (typically small black circles).

An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag.

Would Display

This Code

<pre> Coffee Milk </pre>	• Coffee • Milk
---	---

Ordered Lists

An ordered list is also a list of items. The list items are marked with numbers. An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag.

This Code	Would Display
<pre> Coffee Milk </pre>	• Coffee • Milk

Inside a list item you can put paragraphs, line breaks, images, links, other lists, etc.

Definition Lists

Definition lists consist of two parts: a **term** and a **description**. To mark up a definition list, you need three HTML elements; a container `<dl>`, a definition term `<dt>`, and a definition description `<dd>`.

This Code	Would Display
------------------	----------------------

<pre> <dl> <dt>Cascading Style Sheets</dt> <dd>Style sheets are used to provide presentational suggestions for documentsmarked up in HTML. </dd> </dl> </pre>	Cascading Style Sheets Style sheets are used to provide presentational suggestions for documents marked up in HTML.
---	---

Inside a definition-list definition (the <dd> tag) you can put paragraphs, line breaks, images, links, other lists, etc

Try It Out

Open your text editor and type the following:

```

<html>
<head>
<title>My First Webpage</title>
</head>
<body bgcolor="#EDDD9E">
<h1 align="center">My First Webpage</h1>
<p>Welcome to my <strong>first</strong> webpage. I am writing this page using
a texteditor and plain oldhtml.</p>
<p>By learning html, I'll be able to create web pages like a pro.....<br>
which I am of
course.</p>
Here's what
I've learned:
<ul>
<li>How to use HTML tags</li>
<li>How to use HTML colors</li>
<li>How to create Lists</li>
</ul>
</body>
</html>

```

Save your page as **mypage4.html** and view it in your browser. To see how your page should look visitthisweb page:

<http://profdevtrain.austincc.edu/html/mypage4.html>

1.8.9 HTML Links

HTML uses the <a>anchor tag to create a link to another document or web

page. The Anchor Tag and the Href Attribute

An anchor can point to any resource on the Web: an HTML page, an image, a sound file, a movie, etc. The syntax of creating an anchor:

```
<a href="url">Text to be displayed</a>
```

The <a> tag is used to create an anchor to link from, the href attribute is used to tell the address of the document or page we are linking to, and the words between the open and close of the anchor tag will be displayed as a hyperlink.

This Code	Would Display
<pre> HYPERLINK "http://www.austincc.edu/"Visit HYPERLINK "http://www.austincc.edu/" ACC!</pre>	Visit HYPERLINK "http://www.austincc.edu/ /"_ HYPERLINK "http://www.austincc.edu/"A CC !

The Target Attribute

With the target attribute, you can define **where** the linked document will be opened. By default, the link will open in the current window. The code below will open the document in a new browser window:

```
<a href=http://www.austincc.edu/ HYPERLINK  
"http://www.austincc.edu/" target="_blank">Visit ACC!</a>
```

Email Links

To create an email link, you will use mailto: plus your email address. Here is a link to ACC's Help Desk:

```
<a href= HYPERLINK  
"mailto:helpdesk@austincc.edu"_mailto:helpdesk@austincc.edu HYPERLINK  
"mailto:helpdesk@austincc.edu">Email Help Desk</a>
```

To add a subject for the email message, you would add ?subject= after the email address. For

example:

```
<a href= HYPERLINK
```

<mailto:helpdesk@austincc.edu>

[HYPERLINK "mailto:helpdesk@austincc.edu"?subject=Email
HYPERLINK "mailto:helpdesk@austincc.edu" _Assistance">Email Help](mailto:helpdesk@austincc.edu?subject=Email_Assistance)

DeskThe Anchor Tag and the Name Attribute

The name attribute is used to create a named anchor. When using named anchors we can create links that can jump directly to a specific section on a page, instead of letting the user scroll around to find what he/she is looking for. Unlike an anchor that uses href, a named anchor doesn't change the appearance of the text (unless you set styles for that anchor) or indicate in any way that there is anything special about the text. Below is the syntax of a named anchor:

`<a name="top">Text to be displayed</a>`

1.8.10 HTML Images

The Image Tag and the Src Attribute

The `<img>` tag is empty, which means that it contains attributes only and it has no closing tag. To display an image on a page, you need to use the src attribute. Src stands for "source". The value of the src attribute is the URL of the image you want to display on your page. The syntax of defining an image:

This Code	Would Display
<code></code>	

Not only does the source attribute specify what image to use, but where the image is located. The Above Image, graphics/chef.gif, means that the browser will look for the image name **chef.gif** in graphics

The Alt Attribute

The alt attribute is used to define an alternate text for an image. The value of the alt attribute is author-defined text:

``

The alt attribute tells the reader what he or she is missing on a page if the browser can't load images. The browser will then display the alternate text instead of the image. It is a good practice to include the alt attribute for each image on a page, to improve the display and usefulness of your document for people who have text-only

browsers or use screen readers.

Image Dimensions

When you have an image, the browser usually figures out how big the image is all by itself. If you put in the image dimensions in pixels however, the browser simply reserves a space for the image, then loads the rest of the page. Once the entire page is loaded it can go back and fill in the images. Without dimensions, when it runs into an image, the browser has to pause loading the page, load the image, then continue loading the page. The chef image would then be:

```
Open the file **mypage2.html** in your

text editor and add code highlighted in bold:

```
<html>
<head>
<title>My First Webpage</title>
</head>
<body>
<h1 align="center">My First Web page</h1>
<p>Welcome to my first webpage. I am writing this page using a text editor and plain
old html.</p>
<p>By learning html, I'll be able to create web pages like a
pro.....
 which I am of course.</p>
<!-- Who would have guessed how easy this would be :) -->
<p></p>
<p align="center">This is my Chef</p>
</body>
</html>
```

Save your page as **mypage5.html** and view it in your browser. To see how your page should look visit this web page:

<http://profdevtrain.austincc.edu/html/mypage5.html>

### **1.8.11 HTML Tables**

Tables are defined with the <table>tag. A table is divided into rows (with the <tr>tag), and each row is divided into data cells (with the <td> tag). The letters td stands for table data, which is the content of a data cell. A data cell can contain text, images, lists, paragraphs, forms, horizontal rules, tables, etc.

### **This Code Would Display**

<pre> &lt;table&gt; &lt;tr&gt; &lt;td&gt;row 1, cell 1&lt;/td&gt; &lt;td&gt;row 1, cell 2&lt;/td&gt; &lt;/tr&gt; &lt;tr&gt; &lt;td&gt;row 2, cell 1&lt;/td&gt; &lt;td&gt;row 2, cell 2&lt;/td&gt; &lt;/tr&gt; &lt;/table&gt; </pre>	row 1, cell 1 row 1, cell 2 row 2, cell 1 row 2, cell 2
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	------------------------------------------------------------

### Tables and the Border Attribute

To display a table with borders, you will use the border attribute.

This Code	Would Display
-----------	---------------

<pre> &lt;table border="1"&gt; &lt;tr&gt; &lt;td&gt;Row 1, cell 1&lt;/td&gt; &lt;td&gt;Row 1, cell 2&lt;/td&gt; &lt;/tr&gt; &lt;/table&gt; </pre>	row 1, cell 1 row 1, cell 2
---------------------------------------------------------------------------------------------------------------------------------------------------	-----------------------------

and....

This Code	Would Display
<pre> &lt;table border="5"&gt; &lt;tr&gt; &lt;td&gt;Row 1, cell 1&lt;/td&gt; &lt;td&gt;Row 1, cell 2&lt;/td&gt; &lt;/tr&gt; &lt;/table&gt; </pre>	row 1, cell 1 row 1, cell 2

Open up your text editor. Type in your <html>, <head>and <body>tags. From here on I will only be writing what goes between the <body>tags. Type in the following:

```

<table border="1">
<tr>
<td>Tables can be used to layout information</td>
<td> <img src= HYPERLINK

26


```

<pre> &lt;table border="1"&gt; &lt;tr&gt; &lt;td&gt;some text&lt;/td&gt; &lt;td&gt;some text&lt;/td&gt; &lt;/tr&gt; &lt;tr&gt; &lt;td&gt;some text&lt;/td&gt; &lt;td&gt;some text&lt;/td&gt; &lt;/tr&gt; &lt;/table&gt; </pre>	
--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------	--

```

tml /graphics/chef.gif HYPERLINK
"http://profdevtrain.austincc.edu/html/graphics/chef.gif" _ HYPERLINK
"http://profdevtrain.austincc.edu/html/graphics/chef.gif" ≥ HYPERLINK
"http://profdevtrain.austincc.edu/html/graphics/chef.gif"
</td>
</tr>
</table>

```

Save your page as **mytable1.html** and view it in your browser. To see how your page should look visit this web page:

<http://profdevtrain.austincc.edu/html/mytable1.html>

### Headings in a Table

Heading	Another Heading
row 1, cell 1 r	
row 2, cell 1 r	

Headings in a table are defined with the <th>tag.

### Cell Padding and Spacing

The <table> tag has two attributes known as cellspacing and cellpadding. Here is a table example without these properties. These properties may be used separately or together.

This Code	Would Display
-----------	---------------

some text		some text	
some text		some text	

**Cellspacing** is the pixel width between the individual data cells in the table (The thickness of the lines making the table grid). The default is zero. If the border is set at 0, the cellspacing lines will be invisible.

This Code	Would Display		
<pre>&lt;table border="1" cellspacing="5"&gt; &lt;tr&gt; &lt;td&gt;some text&lt;/td&gt; &lt;td&gt;some text&lt;/td&gt; &lt;/tr&gt;&lt;tr&gt; &lt;td&gt;some text&lt;/td&gt; &lt;td&gt;some text&lt;/td&gt; &lt;/tr&gt; &lt;/table&gt;</pre>			

some text	some text
-----------	-----------

some text	some text
-----------	-----------

Cellpadding is the pixel space between the cell contents and the cell border. The default for this property is also zero. This feature is not used often, but sometimes comes in handy when you have your borders turned on and you want the contents to be away from the border a bit for easy viewing. Cellpadding is invisible, even with the border property turned on. Cellpadding can be handled in a stylesheet.

### **This Code Would Display**

```
<table border="1" cellpadding="10">
<tr>
<td>some text</td>
<td>some text</td>
</tr><tr>
<td>some text</td>
<td>some text</td>
</tr>
</table>
```

### **Table Tags**

Tag	Description
<table>	Defines a table
<th>	Defines a table header
<tr>	Defines a table row
<td>	Defines a table cell
<caption>	Defines a table caption
	Defines groups of table columns
<col>	Defines the attribute values for one or more columns in a table

### **Table Size**

#### Table Width -

The width attribute can be used to define the width of your table. It can be defined as a fixed width or a relative width. A fixed table width is one where the width of the table is specified in pixels. For example, this code, `<table width="550">`, will produce a table that is 550 pixels wide. A relative table width is specified as a percentage of the width of the visitor's viewing window. Hence this code, `<table width="80%">`, will produce a table that occupies 80 percent of the screen.

There are arguments in favor of giving your tables a relative width because such table widths yield pages that work regardless of the visitor's screen resolution. For example, a table width of 100% will always span the entire width of the browser window whether the visitor has a 800x600 display or a 1024x768 display (etc). Your visitor never needs to scroll horizontally to read your page, something that is regarded by most people as being very annoying.

## **Lecture 6**

## 1.8.12 HTML Frames

HTML frames are used to divide your browser window into multiple sections where each section can load a separate HTML document. A collection of frames in the browser window is known as a frameset. The window is divided into frames in a similar way the tables are organized: into rows and columns.

### Disadvantages of Frames

There are few drawbacks with using frames, so it's never recommended to use frames in your webpages

- Some smaller devices cannot cope with frames often because their screen is not big enough to be divided up.
- Sometimes your page will be displayed differently on different computers due to different screen resolution.
- The browser's *back button* might not work as the user hopes. There are still few browsers that do not support frame technology.

### Creating Frames

To use frames on a page we use `<frameset>` tag instead of `<body>` tag. The `<frameset>` tag defines how to divide the window into frames. The **rows** attribute of `<frameset>` tag defines horizontal frames and **cols** attribute defines vertical frames. Each frame is indicated by `<frame>` tag and it defines which HTML document shall open into the frame.

### Example

Following is the example to create three horizontal frames:

```
<!DOCTYPE html>
<html>
<head>
<title>HTML Frames</title>
</head>
<frameset rows="10%,80%,10%">
<frame name="top" src="/html/top_frame.htm" />
<frame name="main" src="/html/main_frame.htm" />
<frame name="bottom" src="/html/bottom_frame.htm" />
</frameset>
<body>
Your browser does not support frames.
</body>
</frameset>
</html>
```

This will produce following result:



Top Frame

Main Frame

Bottom Frame

---

### Browser Support for Frames

- If a user is using any old browser or any browser which does not support frames then `<noframes>` element should be displayed to the user.
- So you must place a `<body>` element inside the `<noframes>` element because the `<frameset>` element is
- supposed to replace the `<body>` element, but if a browser does not understand
- `<frameset>` element then it should understand what is inside the `<body>` element which is contained in a `<noframes>` element.

You can put some nice message for your user having old browsers. For example *Sorry!! your browser does not support frames.* as shown in the above example.

### Frame's name and target attributes

One of the most popular uses of frames is to place navigation bars in one frame and then load main pages into a separate frame.

Let's see following example where a test.htm file has following code:

```

<!DOCTYPE html>
<html>
<head>
<title>HTML Target Frames</title>
</head>
<frameset cols="200, *">
<frame src="/html/menu.htm" name="menu_page" />
<frame src="/html/main.htm" name="main_page" />
<noframes>
<body>
Your browser does not support frames.
</body>
</noframes>
</frameset>
</html>

```

Here we have created two columns to fill with two frames. The first frame is 200 pixels wide and will contain the navigation menu bar implemented by **menu.htm** file. The second column fills in remaining space and will contain the main part of the page and it is implemented by **main.htm** file. For all the three links available in menu bar, we have mentioned target frame as **main\_page**, so whenever you click any of the links in menu bar, available link will open in main\_page. Following is the content of menu.htm file

```

<!DOCTYPE html>
<html>
<body bgcolor="#4a7d49">
Google

Microsoft

BBC News
</body>
</html>

```

## HTML iframes

- HTML Iframe is used to display a nested webpage (a webpage within a webpage). The HTML <iframe> tag defines an inline frame, hence it is also called an Inline frame.
- An HTML iframe embeds another document within the current HTML document in the rectangular region.
- The webpage content and iframe contents can interact with each other using JavaScript.
- Iframe Syntax:
- An HTML iframe is defined with the <iframe> tag:
- <iframe src="URL"></iframe>
- Here, "src" attribute specifies the web address (URL) of the inline frame page.
- Set Width and Height of iframe



You can set the width and height of the iframe by using "width" and "height" attributes.

By default, the attribute values are specified in pixels but you can also set them in percent. i.e. 50%, 60% etc.

Example: (Pixels)

```
<!DOCTYPE html>
<html>
<body>
<h2>HTML Iframes example</h2>
<p>Use the height and width attributes to specify the size of the iframe:</p>
<iframe src="https://www.javatpoint.com/" height="300" width="400"></iframe>
</body>
</html>
```

### 1.8. 13 HTML Forms

HTML Forms use the <form> tag to collect user input through various interactive controls. These controls range from text fields, numeric inputs, and email fields to password fields, checkboxes, radio buttons, and submit buttons.

~These controls range from text fields, numeric inputs, email fields, password fields, to checkboxes, radio buttons, and submit buttons. In essence, an HTML Form serves as a versatile container for numerous input elements, thereby enhancing user interaction.

**Syntax:**

```
<form>
<!--form elements-->
</form>
```

#### Form Elements

The HTML <form> comprises several elements, each serving a unique purpose. For instance, the <label> element is used to define labels for other <form> elements. The <input> element, on the other hand, is versatile and can be used to capture various types of input data such as text, password, email, and more, simply by altering its type attribute. Now, let's all the list of HTML Form Elements one by one:

Elements	Descriptions
<u><a href="#">&lt;label&gt;</a></u>	It defines labels for <form> elements.
<u><a href="#">&lt;input&gt;</a></u>  <u><a href="#">&lt;button&gt;</a></u>	It is used to get input data from the form in various types such as text, password, email, etc by changing its type.   It defines a clickable button to control other elements or execute a functionality.

<u><a href="#">&lt;select&gt;</a></u>	It is used to create a drop-down list.
<u><a href="#">&lt;textarea&gt;</a></u>	It is used to get input long text content.
<u><a href="#">&lt;fieldset&gt;</a></u>  <u><a href="#">&lt;legend&gt;</a></u>  <u><a href="#">&lt;datalist&gt;</a></u>	It is used to draw a box around other form elements and group the related data.   It defines a caption for fieldset elements   It is used to specify pre-defined list options for input controls.
<u><a href="#">&lt;output&gt;</a></u>	It displays the output of performed calculations.
<u><a href="#">&lt;option&gt;</a></u>	It is used to define options in a drop-down list.

<u><a href="#">&lt;optgroup&gt;</a></u>	It is used to define group-related options in a drop-down list.
-----------------------------------------	-----------------------------------------------------------------

### **Commonly Used Input Types in HTML Forms**

In HTML forms, various input types are used to collect different types of data from users. Here are some commonly used input types:

`<input type="text">`

Input Type	Description
<u><a href="#">&lt;input type="text"&gt;</a></u>	Defines a one-line text input field
<u><a href="#">&lt;input type="password"&gt;</a></u>	Defines a password field
<u><a href="#">&lt;input type="submit"&gt;</a></u>	Defines a submit button
<u><a href="#">&lt;input type="reset"&gt;</a></u>	Defines a reset button
<u><a href="#">&lt;input type="radio"&gt;</a></u>	Defines a radio button

<u><code>&lt;input type="email"&gt;</code></u>	Validates that the input is a valid email address.
<u><code>&lt;input type="number"&gt;</code></u>  <u><code>&lt;input</code></u>  <u><code>type="checkbox"&gt;</code></u>  <u><code>&lt;input type="date"&gt;</code></u>	Allows the user to enter a number. You can specify min, max, and step attributes for range.   Used for checkboxes where the user can select multiple options.   Allows the user to select a date from a calendar.
<u><code>&lt;input type="time"&gt;</code></u>	Allows the user to select a time.

<code>&lt;input type="file"&gt;</code>	Allows the user to select a file to upload.
----------------------------------------	---------------------------------------------

### Basic HTML Forms Example:

**Example:** This HTML forms collects the user personal information such as username and password with the button to submit the form.

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Html Forms</title>
</head>
<body>
<h2>HTML Forms</h2>
<form>
<label for="username">Username:</label>

<input type="text" id="username" name="username">

<label for="password">Password:</label>

<input type="password" id="password" name="password">

<input type="submit" value="Submit">
</form>
</body>
</html>
```

### Features:

- Facilitates user input collection through various elements.
- Utilizes <form> tags to structure input elements.
- Defines actions for data submission upon form completion.
- Supports client-side validation for enhanced user experience.

## HTML Coding Practices :

### 1. Use Proper Document Structure With Doctype

HTML has a nature that will still render your markup correctly even if you forget to mention some elements such as `<html>`, `<head>`, `<body>`, and `<!DOCTYPE html>`. You will see the correct result in your browser as you want but that doesn't mean you will find the same result in every browser. To avoid this issue it's a good habit to follow a proper document structure with the correct doctype. Doctype is the first thing to mention in an HTML document. You can choose the right doctype from the importance of doctype.

#### Example:

HTML

```
<!DOCTYPE html>
<html>
 <head>
 <title>Hello World</title>
 </head>
 <body>
 <h1>Welcome Programmers</h1>

 <p>This website is XYZ.</p>

 </body>
</html>
```

### 2. Close the Tags

To avoid validation and compatibility issues don't forget to close all the tags in your code. Today most text editors come up with features that close the HTML tags automatically still, it's a good practice (and definitely for final check) to make sure that you don't miss any parent or nested tag that is not closed. In HTML5 it's optional to close HTML tags but according to W3C specification, you should close all the HTML tags to avoid any validation error in the future.

**Note:** Not all the tags have closing tags, Please check [Which tags contain both opening & closing tags in HTML](#). of using

#### Example:

HTML

```
<div>
 <div>
 <div>
 <p>Hello Programmers</p>

 Array
 Linked List
 Stack

 </div>
 </div>
</div>
```



`<div>` Missing close div -->

### 3. Write Tags in Lowercase

Make a habit of using lowercase for all the tags, attributes, and values in HTML code. It is an industry-standard practice and it also makes your code much more readable. Capitalizing the tags won't affect the result in your browser but it's a good practice to write elements in lowercase. Writing your code in lowercase is easy and it also looks cleaner.

**Example:**

HTML

```
<!-- Wrong practice-->
<SECTION>
 <p>This is a paragraph.</p>
</SECTION>
<!-- Right practice-->
<section>
 <p>This is a paragraph.</p>
</section>
```

### 4. Add Image Attributes

When you add an image to your HTML code don't forget to add an alt attribute for validation and accessibility reasons. Also, make sure that you choose the right description for the alt attribute. Search engines rank your page lower as a result when you don't mention alt attributes with an image. It's also good to mention the height and width of the image. This allows the browser to reserve the appropriate space for the image before it loads, reducing layout shifting and flickering as the page renders.

**Example:**

HTML

```
<!-- Wrong Practice-->

<!-- Right Practice-->

```

### 5. Avoid Using Inline Styles

A lot of newbies make the mistake they adding inline style in HTML tags. It makes your code complicated to maintain. Make a habit of keeping your styles separate from HTML markup. Using inline styles can create so many problems. It makes your code cluttered, unreadable, and hard to update or maintain. Separating HTML from CSS also helps other developers to make changes in code very easily.

**Example:**

HTML

```
<!-- Wrong Practice -->
```

```
<p style="color: #393; font-size: 24px;">Thank you!</p>
```

```
<!-- Right Practice -->
```

```
<p class="alert-success">Thank you!</p>
```

## 6. Use a Meaningful Title and Descriptive Meta Tags

HTML title matters a lot when it comes to search engine optimization or ranking of your page. Always try to make your title as meaningful as possible. The title of your HTML page appears in the Google search engine result page and the indexing of your site relies on it.

A meta description tells the user what the page is all about, so make it descriptive and specify the purpose of your website or page. Avoid using repeated words and phrases. When a user types some keyword in the search engine bar that keyword is picked up by the search engine spiders and then it is used to find the relevant page for users based on the matching keyword used in meta tags.

### Example:

HTML

```
<meta charset="UTF-8" />
<meta
 name="viewport"
 content="width=device-width, initial-scale=1, maximum-scale=1"/>
<meta
 name="description"
 content="
 "A Computer Science portal for geeks. It contains well written,
 well thought and well explained computer science and programming
 articles, quizzes and practice/competitive programming/company
 interview Questions."/>
<meta name="theme-color" content="#0f9d58" />
<meta
 property="og:image"
 content="image/png"/>
<meta property="og:image:type" content="image/png" />
<meta property="og:image:width" content="200" />
<meta property="og:image:height" content="200" />
<script src="https://apis.google.com/js/platform.js"></script>
<script src=
 "https://cdnjs.cloudflare.com/ajax/libs/require.js/2.1.14/require.min.js"></script>
<title>XYZ | A computer science portal for geeks</title>
```

## 7. Use Heading Elements Wisely

Heading tags also play a crucial role in making your website search engine-friendly. HTML has 6 different types of heading tags that should be used carefully. Learn to use `<h1>` to `<h6>` elements to denote your HTML's content hierarchy also make sure that you use only one h1 per page. In W3C specs it is mentioned that your page content should be described by a single tag so you can add your most important text within h1 such as the article title or blog post.

### Example:

## HTML

```
<h1>Topmost heading</h1>
<h2>Sub-heading underneath the topmost heading.</h2>
<h3>Sub-heading underneath the h2 heading.</h3>
```

### 8. Always Use the Right HTML Elements

A lot of beginners make the common mistake of using the wrong HTML elements. They need to learn with time which element should be used where. We recommend you learn about all the HTML elements and use them correctly for a meaningful content structure. For example, instead of using `<br>` between two paragraphs, you can use CSS margin and/or padding properties. Learn when to use `<em>` and when to use `<i>` (both look the same), and learn when to use `<b>` and when to use `<strong>` (both look the same). It comes with practice and when you keep your eyes on good code written by other developers. Another good example is given below.

**Note:** To check the Difference between `<i>` and `<em>` tagstags, [strong and bold tag](#) in HTML

```
<!-- Wrong Practice -->
Hello Geeks

This is Computer Science portal for geeks.

<!-- Right Practice -->
<h1>Hello Geeks</h1>
<p>This is Computer Science portal for geeks.</p>
```

### 9. Proper Use of Indentation

It is important to give proper space or indentation in your HTML code to make it more readable and manageable. Avoid writing everything in one line. It makes your code cluttered and flat. When you use the nested element give proper indentation so that users can identify the beginning and end of the tag. A code that follows proper formatting is easy to change and looks nice to other developers. It's a good coding practice that reduces development time as well.

## HTML

```
<!-- Bad Code -->
<aside>
<h3>XYZ</h3>
<h5>A computer science portal for geeks</h5>

Computer Science
Gate

</aside>

<!-- Good Code -->
<aside>
```

```
<h3>XYZ</h3>
<h5>A computer science portal for geeks</h5>

 Computer Science
 Gate

</aside>
```

## 10. Validate Your Code

Last but not least make a habit to validate your HTML code frequently. Validating your code is a great piece of work and helps in finding difficult issues. You can download the **W3C markup validation** or **Firefox developers toolbar** to validate your site by URL. Validators are very helpful in detecting errors and resolving them.

### Lecture -7

## 1.9 XML Basics

XML stands for **Extensible Markup Language** and is a text-based markup language derived from Standard Generalized Markup Language (SGML).

XML is a software- and hardware-independent tool for storing and transporting data.

- XML is a markup language much like HTML
- XML was designed to store and transport data
- XML was designed to be self-descriptive
- XML is a W3C Recommendation

XML tags identify the data and are used to store and organize the data, rather than specifying how to display it like HTML tags, which are used to display the data. XML is not going to replace HTML in the near future, but it introduces new possibilities by adopting many successful features of HTML.

There are three important characteristics of XML that make it useful in a variety of systems and solutions:

**XML is extensible:** XML allows you to create your own self-descriptive tags, or language, that suits your application.

**XML carries the data, does not present it:** XML allows you to store the data irrespective of how it will be presented.

**XML is a public standard:** XML was developed by an organization called the World Wide Web Consortium (W3C) and is available as an open standard.

## 1.9.1 XML Usage

A short list of XML usage says it all:

- XML can work behind the scenes to simplify the creation of HTML documents for web sites.
- XML can be used to exchange information between organizations and systems. XML can be used for offloading and reloading of databases.
- XML can be used to store and arrange the data, which can customize your data handling needs. XML can easily be merged with style sheets to create almost any desired output. Virtually, any type of data can be expressed as an XML document

## 1.9.2 The Difference between XML and HTML

XML and HTML were designed with different goals:

XML was designed to carry data - with focus on what data is

HTML was designed to display data - with focus on how data looks

XML tags are not predefined like HTML tags

XML documents form a tree structure that starts at "the root" and branches to "the leaves".

## 1.9.3 XML Syntax:

Following is a complete XML document:

```
<?xml version="1.0"?>
<contact_info>
<name>Rajesh</name>
<company>TCS</company>
<phone>9333332354</phone>
</contact_info>
```

You can notice there are two kinds of information in the above example:

markup, like *<contact-info>* and

the text, like Rajesh etc.

### XML Declaration

The XML document can optionally have an XML declaration. It is written as below: `<?xml`

`version="1.0" encoding="UTF-8"?>`

Where *version* is the XML version and *encoding* specifies the character encoding used in the document.

### Syntax Rules for XML declaration

- The XML declaration is case sensitive and must begin with "`<?xml>`" where "xml" is written in lower-case.

- If a document contains an XML declaration, then it strictly needs to be the first statement of the XML document.
- The XML declaration strictly needs to be the first statement in the XML document.
- An HTTP protocol can override the value of *encoding* that you put in the XML declaration.

## Tags and Elements

An XML file is structured by several XML-elements, also called XML-nodes or XML- tags. XML elements' names are enclosed by triangular brackets < > as shown below:

### Syntax Rules for Tags and Elements

**Element Syntax:** Each XML-element needs to be closed either with start or with end elements as shown below:

```
<element>.....</element>
```

or in simple-cases, just this way:

```
<element/>
```

### Nesting of elements:

An XML-element can contain multiple XML-elements as its children, but the children elements must not overlap. i.e., an end tag of an element must have the same name as that of the most recent unmatched start tag.

Following example shows incorrect nested tag

```
<?xml version="1.0"?>
<contact_info>
<company>TCS
<contact_info>
</company>
```

Following example shows correct nested tags:

```
<?xml version="1.0"?>
<contact_info>
<company>TCS</company>
</contact_info>
```

### Root element:

An XML document can have only one root element. For example, following is not a correct

XML document, because both the x and y elements occur at the top level without a root element:

```
<x>...</x>
```

```
<y>...</y>
```

The following example shows a correctly formed XML document:

```
<root>
```

```
<x>...</x>
```

```
<y>...</y>
```

```
</root>
```

### Case sensitivity:

The names of XML-elements are case-sensitive. That means the name of the start and the end elements need to be exactly in the same case.

For example `<contact_info>` is different from `<Contact_Info>`. **Attributes**

An **attribute** specifies a single property for the element, using a name/value pair. An XML-element can have one or more attributes. For example:

```
Tutorialspoint!
```

Here *href* is the attribute name and *http://www.tutorialspoint.com/* is attribute value.

### Syntax Rules for XML Attributes

Attribute names in XML (unlike HTML) are case sensitive. That is, *HREF* and *href* are considered two different XML attributes.

Same attribute cannot have two values in a syntax. The following example shows incorrect syntax because the attribute *b* is specified twice:

```
...
```

Attribute names are defined without quotation marks, whereas attribute values must always appear in quotation marks. Following example demonstrates incorrect xml syntax:

```
...
```

 In the above syntax, the attribute value is not defined in quotation marks.

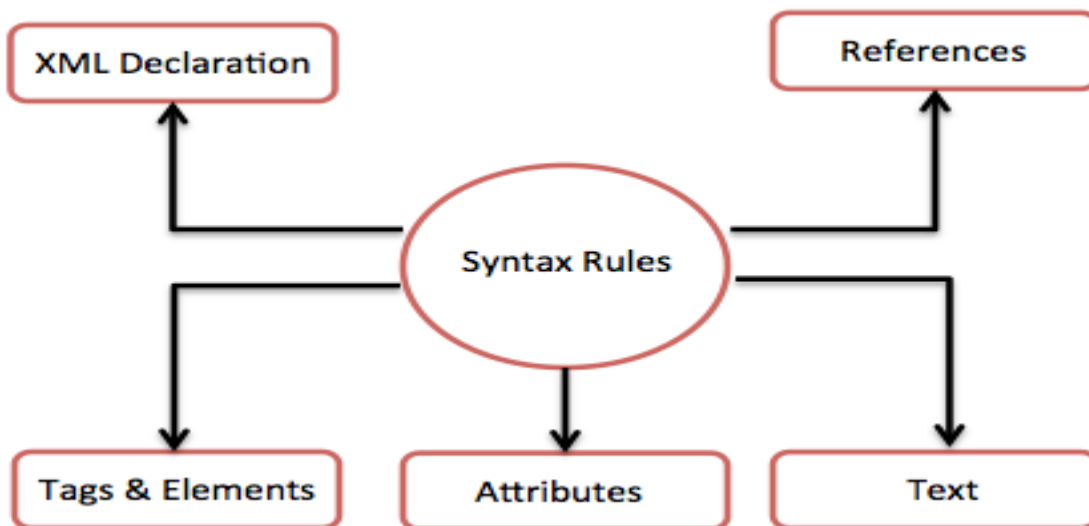


Figure 1.2 XML Rules

### XML References

*References* usually allow you to add or include additional text or markup in an XML document. References always begin with the symbol "&" ,which is a reserved character and end with the symbol ";". XML has two types of references:

**Entity References:** An entity reference contains a name between the start and the end delimiters. For example **&amp;**; where *amp* is the name. The *name* refers to a predefined string of text and/or markup.

**Character References:** These contain references, such as **&#65;**, contains a hash mark ("#") followed by a number. The number always refers to the Unicode code of a character. In this case, 65 refers to the alphabet "A".

### XML Text

The names of XML-elements and XML-attributes are case-sensitive, which means the name of start and end elements need to be written in the same case.

To avoid character encoding problems, all XML files should be saved as UnicodeUTF-8 or UTF-16 files.

Whitespace characters like blanks, tabs and line-breaks between XML-elements and between the XML-attributes will be ignored.

Some characters are reserved by the XML syntax itself. Hence, they cannot be used directly. To use them, some replacement-entities are used, which are listed below:



not allowed character	replacement-entity	character description
<	&lt;	less than
>	&gt;	greater than
&	&amp;	Ampersand
'	&apos;	Apostrophe
"	&quot;	quotation mark

### XML Tree Structure: An Example XML Document

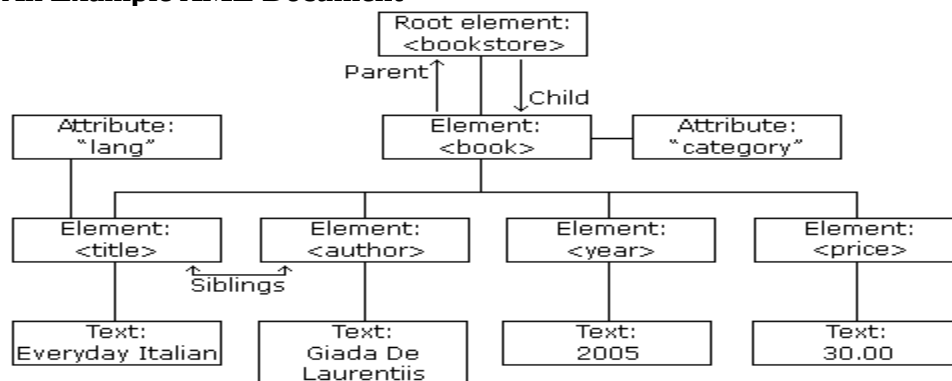


Figure 1.3 XML Tree

The image above represents books in this XML:

```

<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
<book category="cooking">
<title lang="en">Everyday Italian</title>
<author>Giada De Laurentiis</author>
<year>2005</year>
<price>30.00</price>
</book>

```

```

<book category="children">
<title lang="en">Harry Potter</title>
<author>J K. Rowling</author>
<year>2005</year>
<price>29.99</price>
</book>
<book category="web">
<title lang="en">Learning XML</title>
<author>Erik T. Ray</author>
<year>2003</year>
<price>39.95</price>
</book>
</bookstore>

```

XML documents are formed as **element trees**.

An XML tree starts at a **root element** and branches from the root to **child elements**. All elements can have sub elements (child elements):

```

<root>
<child>
<sub child>. </sub child>
</child>
</root>

```

The terms parent, child, and sibling are used to describe the relationships between elements.

Parents have children. Children have parents. Siblings are children on the same level (brothers and sisters).

### Example-XML document for student information

```

<?xml version = "1.0"?>
<!--student1.xml-->
<students>
<student>
<name>
<firstname> James </firstname>
<lastname> Smith </lastname>
</name>
<address>
<street> 101 South Street</street>
<city> Halifax </city>
<email> james@dal.ca </email>
<phone> 4940001 </phone>
</address>
</student>
<student>
<name>
<firstname> Tom </firstname>

```

```

<lastname> White </lastname>
</name>
<address>
<street> 202 Victoria Road </street>
<city> Dartmouth </city>
<email> tom@dal.ca</email>
<phone> 4940002 </phone>
</address>
</student>
</students>

```

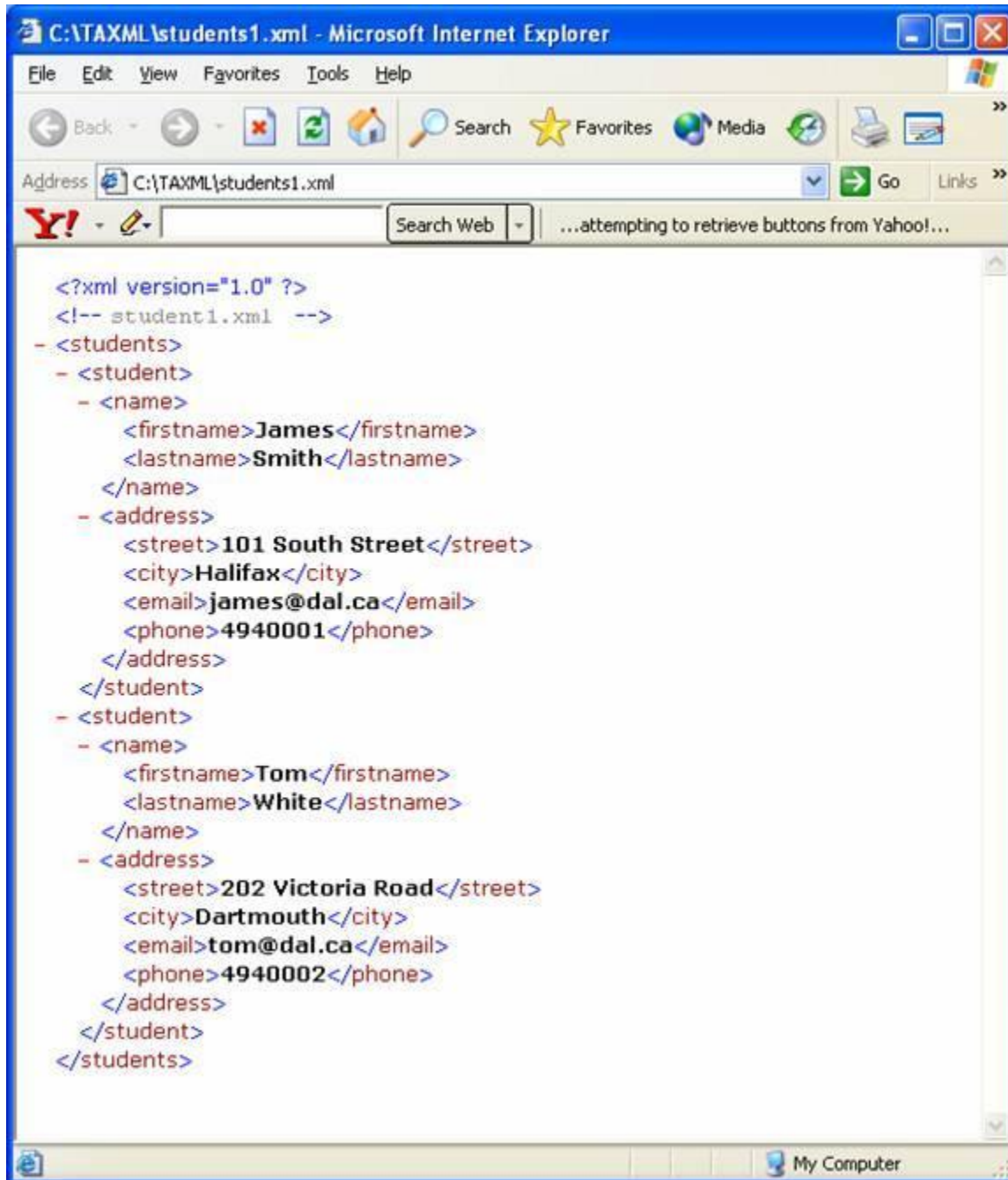


Figure 1.4 XML output

DTD for this is

```

<?xml version = "1.0"?>
<!--students.dtd-a document type definition for the students.xml-->
<!ELEMENT students (student+)>
<!ELEMENT student (name,address)>
<!ELEMENT name (firstname,lastname)>
<!ELEMENT firstname (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>
<!ELEMENT address (street,city,email,phone)>
<!ELEMENT street (#PCDATA)>
<!ELEMENT city (#PCDATA)>
<!ELEMENT email (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>

```

Complete code is

```

<?xml version = "1.0"?>
<!-- students2.xml for the DTD -->
<!DOCTYPE students SYSTEM "students.dtd">
<students>
<student>
<name>
<firstname> James </firstname>
<lastname> Smith </lastname>
</name>
<address>
<street> 101 South Street</street>
<city> Halifax </city>
<email> james@dal.ca </email>
<phone> 4940001 </phone>
</address>
</student>
<student>
<name>
<firstname> Tom </firstname>
<lastname> White </lastname>
</name>
<address>
<street> 202 Victoria Road </street>
<city> Dartmouth </city>
<email> tom@dal.ca</email>
<phone> 4940002 </phone>
</address>
</student>
</students>

```

## XML Namespaces

### Name Conflicts

In XML, element names are defined by the developer. This often results in a conflict when trying to mix XML documents from different XML applications.

This XML carries HTML table information:

```
<table>
<tr>
<td>Apples</td>
<td>Bananas</td>
</tr>
</table>
```

This XML carries information about a table (a piece of furniture):

```
<table>
<name>African Coffee Table</name>
<width>80</width>
<length>120</length>
</table>
```

If these XML fragments were added together, there would be a name conflict. Both Contain a `<table>` element, but the elements have different content and meaning.

A user or an XML application will not know how to handle these differences.

### **Solving the Name Conflict Using a Prefix**

Name conflicts in XML can easily be avoided using a name prefix.

This XML carries information about an HTML table, and a piece of furniture:

```
<h:table>
<h:tr>
<h:td>Apples</h:td>
<h:td>Bananas</h:td>
</h:tr>
</h:table>

<f:table>
<f:name>African Coffee Table</f:name>
<f:width>80</f:width>
<f:length>120</f:length>
</f:table>
```

In the example above, there will be no conflict because the two `<table>` elements have different names.

### **XML Namespaces - The xmlns Attribute**

When using prefixes in XML, a **namespace** for the prefix must be defined.

The namespace can be defined by an **xmlns** attribute in the start tag of an element. The

namespace declaration has the following syntax. `xmlns:prefix="URI"`.

```
<root>
<h:table xmlns:h="http://www.w3.org/TR/html4/">
<h:tr>
<h:td>Apples</h:td>
<h:td>Bananas</h:td>
</h:tr>
</h:table>

<f:table xmlns:f="https://www.w3schools.com/furniture">
<f:name>African Coffee Table</f:name>
<f:width>80</f:width>
<f:length>120</f:length>
</f:table>
</root>
```

In the example above:

The `xmlns` attribute in the first `<table>` element gives the `h:` prefix a qualified namespace. The `xmlns` attribute in the second `<table>` element gives the `f:` prefix a qualified namespace.

When a namespace is defined for an element, all child elements with the same prefix are associated with the same namespace.

## XML Validator

Use our XML validator to syntax-check your XML.

## Well Formed XML Documents

- An XML document with correct syntax is called "Well Formed".The syntax rules were described in the previous chapters:
- XML documents must have a root element XML elements must have a closing tag XML tags are case sensitive
- XML elements must be properly nested
- XML attribute values must be quoted

### Example 1:

```
<?xml version="1.0"?>
```

```
<book>
<title>Java</Title>
<author>James</book>
<pirce>570
</author>
```

The above XML document is not a well formed document. Reasons given below... tags are not matching <title> ... </Title>

There is no proper nesting <author>....</book>

Tag doesn't closed <price>

Example 2:

```
<?xml version="1.0"?>
<book>
<title>Java</title>
<author>James</author>
<price>500</price>
</book>
```

The above XML document is a well formed document.

## **Valid XML Documents**

A "well formed" XML document is not the same as a "valid" XML document.

"valid" XML document must be well formed. In addition, it must conform to a document type definition.

There are two different document type definitions that can be used with XML:

DTD - The original Document Type Definition

XML Schema - An XML-based alternative to DTD

A document type definition defines the rules and the legal elements and attributes for an XML document.

### **1.9.4 XML DTD: (Document Type Definition)**

An XML document with correct syntax is called "Well Formed".

An XML document validated against a DTD is both "Well Formed" and "Valid".

A "Valid" XML document is a "Well Formed" XML document, which also conforms to the rules of a DTD.

DTD is the basic building block of XML.

The purpose of a DTD is to define the structure of an XML document. It defines the structure with a list of legal elements.

```
<!DOCTYPE book[
```

```
<!ELEMENT book (title,author,price)>
<!ELEMENT title (#PCDATA)> <!ELEMENT
author(#PCDATA)> <!ELEMENT price
(PCDATA)>]>
```

The DTD above is interpreted like this:

!DOCTYPE book defines that the root element of the document is book

!ELEMENT book defines that the book element must contain the elements: "title, author, price"

!ELEMENT title defines the title element to be of type "#PCDATA"

!ELEMENT author defines the author element to be of type "#PCDATA"

!ELEMENT price defines the price element to be of type "#PCDATA"Note: PCDATA:

Parsable Character Data, CDATA: Character Data. There are two types of DTDs:

1) Internal / Embedded DTD.

2) External DTD.

**1) Internal / Embedded DTD.**

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE student [
<!ELEMENT student (id,name,age,addr,email,ph)>
<!ELEMENT id (#PCDATA)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT age (#PCDATA)>
<!ELEMENT addr (#PCDATA)>
<!ELEMENT email (#PCDATA)>
<!ELEMENT ph (#PCDATA)>]>
```

```
<student>
<id>543</id>
<name>Ravi</name>
<age>21</age>
<addr>Guntur</addr>
<email>nsr@gmail.com</email>
<ph>9855555</ph>
<gender>male</gender>
</student>
```

**2) External DTD.**

```
<!ELEMENT student (id,name,age,addr,email)>
<!ELEMENT id (#PCDATA)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT age (#PCDATA)>
<!ELEMENT addr (#PCDATA)>
<!ELEMENT email (#PCDATA)>
```

Save the above code as "student.dtd" and prepare "student.xml" as follows...



```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE student SYSTEM "student.dtd">
<student>
<id>543</id>
<name>Ravi</name>
<age>21</age>
<addr>Guntur</addr>
<email>nsr@gmail.com</email>
</student>

```

In the above example we are using `<!DOCTYPE student SYSTEM "student.dtd">` which is used to provide “student.dtd” code in our “student.xml” file.

If the above xml code follows the exact rules defined in DTD then we can conclude that our xml document is a valid document. Otherwise it is an invalid document.

#### 1.9.4.1 When to Use a DTD/Schema?

With a DTD, independent groups of people can agree to use a standard DTD for interchanging data.

With a DTD, you can verify that the data you receive from the outside world is valid.

You can also use a DTD to verify your own data.

#### 1.9.5 XML Schema

An XML Schema describes the structure of an XML document, just like a DTD. An XML document with correct syntax is called "Well Formed".

An XML document validated against an XML Schema is both "Well Formed" and "Valid".

XML Schema is commonly known as XML Schema Definition (XSD). It is used to describe and validate the structure and the content of XML data. XML schema defines the elements, attributes and data types. Schema element supports Namespaces. It is similar to a database schema that describes the data in a database.

XML Schema is an XML-based alternative to DTD:

#### Syntax

You need to declare a schema in your XML document as follows:

```

<xs:schema>

<xs:element name="book">
<xs:complexType>
<xs:sequence>
<xs:element name="title" type="xs:string"/>
<xs:element name="author" type="xs:string"/>
<xs:element name="price" type="xs:integer"/>

```

```

<xs:element name="edition" type="xs:string"/>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>

```

The Schema above is interpreted like this:

```

<xs:element name="book"> defines the element called "book" is a root.
<xs:complexType> the "book" element is a complex type i.e. root
<xs:sequence> the complex type is a sequence of elements i.e. childrens
<xs:element name="title" type="xs:string"> the element "title" is of type string (text)
 <xs:element name="author" type="xs:string"> the element "author" is of type string
<xs:element name="price" type="xs:integer"> the element "price" is of type integer.
 <xs:element name="edition" type="xs:string"> the element "edition" is of type string

```

### Example:

```

<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
targetNamespace="http://www.w3schools.com" elementFormDefault="qualified"> <xs:element
name="student">
<xs:complexType>
<xs:sequence>
<xs:element name="name" type="xs:string"/>
<xs:element name="age" type="xs:integer"/>
<xs:element name="addr">
<xs:complexType>
<xs:sequence>
<xs:element name="city" type="xs:string"/>
<xs:element name="pincode" type="xs:long"/>
</xs:sequence>
</xs:complexType>
</xs:element>
<xs:element name="ph" type="xs:integer"/>
</xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>

```

Save the above code as “student.xsd” Prepare

“student.xml” as follows...

```

<?xml version="1.0" encoding="UTF-8"?>
<student xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.w3schools.com student.xsd">
<name>rajesh</name>

```

```
<age>25</age>
<addr>
<city>mylavaram</city>
<pincode>53333</pincode>
</addr>
<ph>9343434</ph>
</student>
```

If the above xml code follows the exact rules defined in “student.xsd” then we can conclude that our xml document is a valid document. Otherwise it is an invalid document.

### **Lecture-8**

#### **1.9.6 XML DOM Parser**

The DOM defines a standard for accessing and manipulating documents:

The HTML DOM defines a standard way for accessing and manipulating HTML documents. It presents an HTML document as a tree-structure.

The XML DOM defines a standard way for accessing and manipulating XML documents. It presents an XML document as a tree-structure.

#### **The HTML DOM**

All HTML elements can be accessed through the HTML DOM.

This example changes the value of an HTML element with id="demo":

#### **Example:**

```
<!DOCTYPE html>
<html>
<body>

<h1 id="demo">This is a Heading</h1>

<button type="button"
onclick="document.getElementById('demo').innerHTML = 'Hello World!'">Click Me!

</button>

</body>
</html>
```

#### **The XML DOM**

All XML elements can be accessed through the XML DOM. The

XML DOM is:

- A standard object model for XML
- A standard programming interface for XML Platform- and language-independent
- A W3C standard In other words: **The XML DOM is a standard for how to get, change, add, delete XML elements.**

## Programming Interface

The DOM models XML as a set of node objects. The nodes can be accessed with JavaScript or other programming languages.

The programming interface to the DOM is defined by a set of standard properties and methods. **Properties** are often referred to as something that is (i.e. nodename is "book"). **Methods** are often referred to as something that is done (i.e. delete "book").

## XML DOM Properties

These are some typical DOM properties:

x.nodeName - the name of x  
x.nodeValue - the value of x  
x.parentNode - the parent node of x  
x.childNodes - the child nodes of x  
x.attributes - the attributes nodes of xNote: In the list above, x is a node object.

## XML DOM Methods

x.getElementsByTagName(*name*) - get all elements with a specified tag name  
x.appendChild(*node*) - insert a child node to x  
x.removeChild(*node*) - remove a child node from xNote: In the list above, x is a node object.

## DOM Example:

```
<html>
<body>

<p id="demo">this is paragraph text</p>
<button type="button" onclick="myfun()">click me</button>

<script>
function myfun()
{
var text, parser, xmlDoc;

text = "<bookstore><book>"
+"<title>Java</title>" +
"<author>James</author>" +
"<year>1991</year>" +

"<book>" +
"<title>DBMS</title>" +
"<author>Raghu</author>" +
"<year>1970</year>" +
```

```
"</book>" +
"</book></bookstore>";
```

```
parser = new DOMParser();
xmlDoc = parser.parseFromString(text, "text/xml");
```

```
document.getElementById("demo").innerHTML =
xmlDoc.getElementsByTagName("year")[0].childNodes[0].nodeValue;
}
</script>
</body>
</html>
```

### Output:

1991

### Example Explained

**xmlDoc** - the XML DOM object created by the parser.

**getElementsByTagName("year")[0]** - get the first <year> element

**childNodes[0]** - the first child of the <year> element (the text node)

**nodeValue** - the value of the node (the text itself)

## 1.9.7 SAX

SAX is a way of reading data from an XML document that is an alternative to the Document Object Model's mechanism (DOM). Whereas the DOM works on the document as a whole, creating the whole abstract syntax tree of an XML document for the user's convenience, SAX parsers work on each element of the XML document sequentially, issuing parsing events while passing through the input stream in a single pass. Unlike DOM, SAX does not have a formal specification.

It is a programming interface for processing XML files based on events. The DOM's counterpart, SAX, has a very different way of reading XML code. The Java implementation of SAX is regarded as the de-facto standard. SAX processes documents state-independently, in contrast to DOM which is used for state-dependent processing of XML documents.

### 1.9.7.1 Why use SAX Parser ?

Parsers are used to process XML documents. The parser examines the XML document, checks for errors, and then validates it against a schema or DTD if it's a validating parser. The next step is determined by the parser in use. It may copy the data into a data structure native to the computer language you're using on occasion. It may also apply styling to the data or convert it into a presentation format.

Apart from triggering certain events, the SAX parser does nothing with the data. It is up to the SAX parser's user to decide. The SAX events include (among others) as follows:

1. XML Text Nodes
2. XML Element Starts and Ends
3. XML Processing Instructions
4. XML Comments

The properties of SAX Parser are depicted below as follows:

<b>SAX Parser</b>	<b>DOM Parser</b>
It is called a Simple API for XML Parsing.	It is called a Document Object Model.
It's an event-based parser.	It stays in a tree structure.
SAX Parser is slower than DOM Parser.	DOM Parser is faster than SAX Parser.
Best for the larger sizes of files.	Best for the smaller size of files.
It is suitable for making XML files in Java.	It is not good at making XML files in low memory.
The internal structure can not be created by SAX Parser.	The internal structure can be created by DOM Parser.
It is read-only.	It can insert or delete nodes.
In the SAX parser backward navigation is not possible.	In DOM parser backward and forward search is possible

### Unit -1 Important & Previous year Questions

1. Define Protocol. Provide the name of protocols governing the web.
2. What is a web project? Explain web development strategies. Also explain Web Team?
3. What is the client server relationship? How is it implemented? How is the client server paradigm different from peer to peer paradigm?
4. Differentiate between the Internet and WWW? What are the different services provided by the Internet?
5. Create an HTML code to create a web page that contains the user registration form with following details: user name, user date of birth, user address, user gender, user email id, user mobile number.
6. What is a Form? Explain method and action attribute of a Form.
7. Differentiate between an Ordered list and an Unordered list?
8. What do you understand about hyperlinks in HTML? Write mark-up for an image hyperlink
9. Write HTML code to design “a form for online Railway Reservation Website” (Make Assumptions). OR Write the HTML code to design a Student registration form. (Assume fields).
10. Discuss a frame and its advantages. Design HTML document that divides the browser screen into two frames. The frame on the left will be a menu consisting of hyperlinks. Clicking on any one of these links will lead to a new page open in the target frame on right hand side. Differentiate between HTML and XML
11. Explain <iframe> and frameset. Explain the anchor and table tag in HTML?
12. Explain Table tag with attributes. Design a XML DTD for self describing weather report having following details: Date, location, temperature range (Location describes city, state and its country. Country code is unique and not left blank. Temperature range describes high and low temp. in Fahrenheit or Celsius)
13. Write an example of XML DTD. What are the rules for writing DTD? Explain with the help of an example of a book catalog details. What is DTD? Discuss its differences with XML Schema.
14. Write suitable XML markup to display the data of books with title, author, publisher and price. Explain well formed and valid xml? Differentiate between SAX and DOM parsers in detail.

## Practice Questions

1. What will be output of the following code.

```
<!DOCTYPE html>
```

```
<html>
```

```
<style>
```

```
table, th, td {
```

```
 border:1px solid black;
```

```
}
```

```
</style>
```

```
<body>
```

```
<h2>TH elements define table headers</h2>
```

```
<table style="width:100%">
```

```
<tr>
```

```
<th>Person 1</th>
```

```
<th>Person 2</th>
```

```
<th>Person 3</th>
```

```
</tr>
```

```
<tr>
```

```
<td>Emil</td>
```

```
<td>Tobias</td>
```

```
<td>Linus</td>
```

```
</tr>
```

```
<tr>
```

```
<td>16</td>
```

```
<td>14</td>
```

```
<td>10</td>
```

```
</tr>
```

```
</table>
```

```
<p>To understand the example better, we have added borders to the table.</p>
```

```
</body>
```

```
</html>
```

2. What will be output of the following code

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
table, th, td {
```

```
 border: 1px solid black;
```

```
 border-collapse: collapse;
```



```
}
</style>
</head>
<body>
```

<h2>Cell that spans two rows</h2>

<p>To make a cell span more than one row, use the rowspan attribute.</p>

```
<table style="width:100%">
 <tr>
 <th>Name</th>
 <td>Jill</td>
 </tr>
 <tr>
 <th rowspan="2">Phone</th>
 <td>555-1234</td>
 </tr>
 <tr>
 <td>555-8745</td>
 </tr>
</table>
</body>
</html>
```

3.What will be output of the following code.

```
<!DOCTYPE html>
<html>
<head>
 <title>table</title>
 <style>
 table{
 border-collapse: collapse;
 width: 100%;
 }
 th,td{
 border: 2px solid green;
 padding: 15px;
 }
 </style>
</head>
<body>
 <table>
 <tr>
 <th>1 header</th>
 <th>1 header</th>
 <th>1 header</th>
 </tr>
```

```
<tr>
 <td>1data</td>
 <td>1data</td>
 <td>1data</td>
</tr>
<tr>
 <td>2 data</td>
 <td>2 data</td>
 <td>2 data</td>
</tr>
<tr>
 <td>3 data</td>
 <td>3 data</td>
 <td>3 data</td>
</tr>
</table>
</body>
</html>
```